

Password Security & Hash Cracking Lab — Full Explainer

This document explains **what** every part of this project does, **why** it exists, and **which commands** you ran at each stage — written so you can explain it confidently to an interviewer, a recruiter, or a friend with zero security background.

1. The Big Idea (explain this first, always)

Every time you create an account somewhere, the website doesn't store your actual password. Instead, it runs your password through a **hash function** — a one-way mathematical process that turns `"password123"` into something like `482c811da5d5b4bc6d497ffa98491e38`. The website stores that scrambled version.

When you log in later, it hashes what you typed and checks if the two scrambled versions match. In theory, this means even if a hacker steals the database, they only get scrambled gibberish, not real passwords.

The catch: not all hashing is equal. Some hashing methods (MD5, SHA-1) are old, fast, and weak. Some (bcrypt) are modern, slow, and strong. This project proves — with real numbers, not just claims — why that difference matters enormously.

That's the whole story you're demonstrating: **weak hashing = passwords crack in seconds. Strong hashing = passwords become practically uncrackable.**

2. Why This Project Matters (for your resume pitch)

Security jobs care about two things:

1. **Offense** — can you find/exploit weaknesses? (This is what "hacking" usually means.)
2. **Defense** — do you understand how to prevent those weaknesses?

This project proves both. You *attacked* weak hashes (cracked them), and you *built* a defensive tool (the strength checker) and *explained* the correct defense (bcrypt). That's a complete security mindset, and it's exactly what makes this project stronger than "I just ran a tool I downloaded."

3. The Environment — Why Kali Linux + VirtualBox

Choice	Why
Kali Linux	A Linux distribution built specifically for security work. It comes with hundreds of pentesting tools pre-installed — including John the Ripper, Hashcat, and a massive real-world password wordlist (<code>rockyou.txt</code>) — so there's zero setup friction.
VirtualBox	Lets you run Kali as a "virtual computer" inside your normal Windows/Mac machine, safely isolated from your real files and network. Nothing you do inside the VM can damage your host computer.
Why not just Windows?	The tools (John, Hashcat) are built Linux-first. Doing this on Windows means fighting installers and missing wordlists. Kali gives you a working lab in minutes, not hours.

4. Project File Structure — What Each File Is For

```
password-lab/
├─ hash_generator.py      ← STAGE 1: creates the weak hashes to attack
├─ strength_checker.py   ← STAGE 4: the defensive tool you built
├─ bcrypt_demo.py        ← STAGE 3: proves WHY strong hashing works
├─ wordlist_sample.txt    ← a small custom dictionary for cracking demos
├─ results/
│   └─ cracking_log.md    ← your raw proof: real terminal output, timings
├─ README.md             ← the front page of your GitHub repo
└─ writeup.md             ← your narrative explanation – the "story"
version
```

Think of it like a science experiment:

- `hash_generator.py` = preparing your test samples
- John the Ripper / Hashcat (external tools) = your lab equipment
- `results/cracking_log.md` = your raw data / lab notes
- `bcrypt_demo.py` = your control experiment (showing what changes with a better method)
- `strength_checker.py` = the practical tool you built from what you learned
- `writeup.md` = your conclusion / lab report
- `README.md` = the summary sheet on the front of your report

5. STAGE 1 — Generating Weak Hashes (`hash_generator.py`)

What it does

Takes a list of common weak passwords (`"password"`, `"123456"`, `"qwerty"`, etc.) and converts each one into an **MD5 hash** and a **SHA-1 hash** using Python's built-in `hashlib` library. It writes these hashes into two text files, plus an answer key showing which hash corresponds to which real password (for your own verification later).

Why MD5 and SHA-1 specifically?

These are two of the **oldest, fastest, and most breakable** hashing algorithms still found in the wild. They were never designed for passwords — they were designed for things like verifying file integrity, where speed is a *feature*. For passwords, speed is a *liability*, because a fast hash means an attacker can test millions of guesses per second.

The command

```
bash  
  
python3 hash_generator.py
```

What happens: Python runs top to bottom. It loops through each password in the `WEAK_PASSWORDS` list, hashes it, and writes the result to `md5_hashes.txt` and `sha1_hashes.txt`. It also writes `answer_key.csv` — a plaintext-to-hash mapping, purely so you can verify your cracking results are correct (never do this outside a lab you control).

Explain it like this to someone else:

"I wrote a script that takes real weak passwords people actually use, and hashes them the same way an old or poorly-built website would. This gives me realistic targets to test cracking tools against — safely, since it's all self-generated, not real stolen data."

6. STAGE 2 — Cracking the Hashes (John the Ripper)

What John the Ripper actually does

John the Ripper is a **password cracking tool**. It doesn't "decrypt" a hash (hashing isn't reversible by design) — instead, it takes every word in a **wordlist** (a giant text file of possible passwords), hashes each one the same way, and checks if the result matches any hash in your target file. If it matches, it's found the original password.

What `rockyou.txt` is

`rockyou.txt` is a real leaked password list from a 2009 data breach of the company RockYou — it contains over **14 million real passwords** that real people actually used. It's the industry-standard wordlist for demonstrating how predictable human passwords are. It comes bundled with Kali, just compressed.

The commands, explained one by one

```
bash
```

```
sudo gunzip /usr/share/wordlists/rockyou.txt.gz
```

- `gunzip` = decompress a `.gz` (gzip) file.
- `sudo` = run as administrator, since this file lives in a system-protected folder.
- **Why needed:** Kali ships this wordlist pre-compressed to save disk space. You only need to do this once.

```
bash
```

```
time john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt md5_hash
```

- `time` = a Linux command that measures how long the following command takes to run — this is how you got your real "0.70 seconds" statistic.
- `john` = launches John the Ripper.
- `--format=raw-md5` = tells John the hashes in your file are plain MD5 (not MD5 with some other wrapping format).
- `--wordlist=/usr/share/wordlists/rockyou.txt` = tells John which dictionary to test against your hashes.
- `md5_hashes.txt` = the file containing the hashes you want cracked.

What happens under the hood: John reads each line of `rockyou.txt`, hashes it with MD5, and compares it against every hash in `md5_hashes.txt`. The moment it finds a match, it prints the cracked password live in your terminal.

```
bash
```

```
john --show --format=raw-md5 md5_hashes.txt
```

- This doesn't crack anything new — it just neatly re-displays everything John has *already* cracked and saved internally (John keeps a "pot file" of cracked results so it doesn't repeat work).

Your actual result

All 10 weak passwords were cracked in **0.70 seconds**. That's the single most powerful sentence in your whole writeup — say it exactly like that in interviews.

Explain it like this to someone else:

"I used John the Ripper with a real leaked 14-million-password wordlist. It cracked all ten of my test passwords in under one second — proving that common, weak passwords aren't really 'protected' by hashing at all if the hash algorithm is fast and unsalted."

7. STAGE 3 — Proving *Why Strong Hashing Wins* (`bcrypt_demo.py`)

What it does

This script runs two experiments back-to-back and times both:

1. Hashes the same password **100,000 times using MD5** and measures total time.
2. Hashes the password **once using bcrypt** and measures that time.

Then it calculates how many times slower bcrypt is per hash, and explains why that's actually a *good* thing for security.

Why speed is the whole point

Imagine an attacker isn't looking up your password from a wordlist — they're brute-forcing, trying every possible combination. If a hash algorithm can be computed 800,000 times per second (like your MD5 result), an attacker can try 800,000 guesses every second. If a hash algorithm can only be computed roughly 2 times per second (like bcrypt), an attacker's brute-force attack becomes **tens of thousands of times slower** — turning an attack that might take minutes into one that would take centuries.

What "salting" means (the other half of the story)

- With MD5/SHA-1, the same password *always* produces the exact same hash, everywhere, every time.
- This means attackers can precompute — once — a giant table mapping common passwords to their hashes (a "rainbow table"), and then instantly look up any stolen hash without cracking anything live.
- bcrypt automatically mixes in a **random salt** — extra random data — before hashing. So the same password produces a *different* hash every single time it's hashed. A precomputed rainbow table becomes useless, because there's no single hash value to look up.

The command

```
bash  
  
python3 bcrypt_demo.py
```

Your actual result

- MD5: ~798,907 hashes computed per second
- bcrypt: 1 hash took 0.45 seconds
- **bcrypt was ~90,075x slower per hash than MD5**

Explain it like this to someone else:

"I benchmarked MD5 against bcrypt directly. MD5 computed nearly 800,000 hashes per second — great for file checksums, terrible for passwords. bcrypt was about 90,000 times slower per hash *by design*, plus it automatically adds a random salt so identical

passwords never produce identical hashes. That's why every modern authentication system uses something like bcrypt instead of MD5."

8. STAGE 4 — Building the Defensive Tool (`strength_checker.py`)

What it does

This is a Python tool that scores any password you give it across three dimensions:

1. **Length** — under 8 characters = weak, 8-11 = okay, 12+ = strong.
2. **Complexity** — checks for lowercase letters, uppercase letters, digits, and symbols, and reports which are missing.
3. **Breach check** — checks whether this exact password has ever appeared in a known public data breach, using the **Have I Been Pwned (HIBP)** API.

The clever privacy trick — "k-anonymity"

You might wonder: "wait, doesn't sending my password to a website defeat the purpose of a *privacy-respecting* checker?" This is the important technical detail:

1. The script hashes your password locally with SHA-1 (this never leaves your machine).
2. It only sends the **first 5 characters** of that hash to the HIBP API.
3. HIBP responds with a list of *all* breached hashes that start with those same 5 characters — often hundreds of them.
4. Your script then checks locally whether *your full hash* is anywhere in that returned list.

The real API never sees your actual password, or even your full hash — just a 5-character prefix shared by hundreds of other unrelated hashes. This is called **k-anonymity**, and it's the same privacy technique real security tools like browser breach-warnings use.

The command

```
bash
python3 strength_checker.py
```

Your actual results — and why they're the most interesting finding in your whole project

Password	Length	Complexity	Breach count	Verdict
<code>password</code>	Adequate	1/4 types	52,372,427	WEAK
<code>P@ssw0rd</code>	Adequate	4/4 types (all!)	6,421,042	WEAK
<code>Tr0ub4dor&3xtra!Long2024</code>	Strong	4/4 types	Not found	STRONG

This table is gold for your writeup, because it disproves a common myth: `P@ssw0rd` *technically* satisfies every traditional "complexity rule" (uppercase, lowercase, digit,

symbol) — yet it's been found in breaches over **6.4 million times**, because everyone thinks of the same "clever" substitutions (`a`→`@`, `o`→`0`). Complexity rules alone don't make a password safe; *uniqueness and length* do.

Explain it like this to someone else:

"I built a strength checker that doesn't just check character rules — it checks real-world breach data through a privacy-safe API. It proved that a 'complex-looking' password like P@sswOrd is still catastrophically weak, because it's been breached over six million times. Meanwhile, a longer, more random passphrase had zero breach hits. This shows why modern security guidance (like NIST's) has moved away from 'complexity rules' toward 'length and uniqueness' instead."

9. STAGE 5 — Documenting Results (`results/cracking_log.md`)

What it's for

This is your **raw evidence file** — the terminal output, timings, and commands, copy-pasted exactly as they happened. Think of it as the appendix of a lab report: unpolished, factual, and provable. Anyone reviewing your GitHub repo (like a hiring manager) can see you didn't fabricate results — the actual command output is right there.

What goes in it

- The exact commands you ran
 - The raw terminal output (cracked passwords, timings)
 - A short "observations" section connecting the dots (e.g., "MD5 and SHA-1 cracked at nearly the same speed because the bottleneck was wordlist matching, not algorithm speed")
-

10. STAGE 6 — `README.md` (the front door of your repo)

Why it matters most

On GitHub, `README.md` is the **first and often only** thing a recruiter or hiring manager reads. If it's unclear, they won't dig further — no matter how good your code is.

What a good README does

1. States what the project is in one sentence.
 2. Explains *why it matters* (what security concept it demonstrates).
 3. Shows the file structure, so a reader can navigate quickly.
 4. Gives exact "how to reproduce this" commands — this proves it's real, working code, not just claims.
 5. States the key takeaway in plain English.
 6. Includes a short disclaimer that this was all done in a self-contained, authorized lab.
-

11. STAGE 7 — `writeup.md` (your narrative — the "human" version)

Why this is different from the README

The README is a technical reference. The writeup is a **story** — written the way you'd explain it to a smart friend who isn't a security expert. It should walk through: the setup → what you found → why it happened → what the fix is → what you learned. This is the piece that shows *understanding*, not just execution — and it's the piece interviewers actually enjoy reading, because it shows you can communicate security concepts clearly, which is a genuinely rare and valuable skill.

12. STAGE 8 — Publishing to GitHub

The commands, explained

```
bash
git init
```

Turns your local `password-lab` folder into a Git repository — Git starts tracking changes to every file inside it.

```
bash
git add .
```

Stages **all** files in the folder to be included in your next "save point" (commit). The `.` means "everything in this folder."

```
bash
git commit -m "Password Security & Hash Cracking Lab"
```

Creates a permanent snapshot of your staged files, with a short message describing what this snapshot represents.

```
bash
git branch -M main
```

Renames your default branch to `main` (the modern GitHub standard, replacing the older default name `master`).

```
bash
git remote add origin https://github.com/YOUR_USERNAME/YOUR_REPO.git
```

Tells Git "this local repo is connected to this specific GitHub repository online" — `origin` is just a nickname for that remote address.

```
bash
```

```
git push -u origin main
```

Uploads your commit to GitHub. The `(-u)` flag remembers this connection, so future pushes can just be `(git push)`.

13. The One-Paragraph Summary (memorize this for interviews)

"I built a lab that demonstrates both attack and defense in password security. On the attack side, I generated weak password hashes using MD5 and SHA-1, then cracked all of them in under a second using John the Ripper and a real 14-million-entry leaked password list — proving how fast weak, unsalted hashing falls apart. On the defense side, I benchmarked bcrypt against MD5 and showed it's roughly 90,000 times slower per hash, plus automatically salted — which is exactly why it resists both brute-force and rainbow table attacks. Finally, I built a password strength checker that checks real breach data through a privacy-safe API, which proved that 'complex-looking' passwords like P@ssw0rd are still extremely weak if they've been breached millions of times. The whole thing runs in an isolated Kali Linux lab I set up myself, and it's fully documented and reproducible on GitHub."

Quick Reference — Every Command in Order

```
bash
```

```
# 1. Setup
pip install bcrypt requests --break-system-packages

# 2. Generate hashes
python3 hash_generator.py

# 3. Prep wordlist (once)
sudo gunzip /usr/share/wordlists/rockyou.txt.gz

# 4. Crack MD5
time john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt md5_hash
john --show --format=raw-md5 md5_hashes.txt

# 5. Crack SHA-1
time john --format=raw-sha1 --wordlist=/usr/share/wordlists/rockyou.txt sha1_ha
john --show --format=raw-sha1 sha1_hashes.txt

# 6. Run the bcrypt comparison
python3 bcrypt_demo.py

# 7. Run the strength checker
python3 strength_checker.py

# 8. Push to GitHub
git init
git add .
git commit -m "Password Security & Hash Cracking Lab"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/YOUR_REPO.git
git push -u origin main
```